UNIDAD 8 / SEMANAS 12 Y 13

Trabajo Práctico N.º 8

Food Store en la feria: la base de la tablet y el cierre de la capa de datos

FOOD STORE | PROYECTO INTEGRADOR

Asignatura	Base de Datos II · Tecnicatura Universitaria en Programación
Tipo de entrega	Último TP de la materia, antes del segundo parcial
Modalidad	Por equipos del proyecto · defensa con preguntas individuales
Entrega	Cierre de la semana 13, en la fecha publicada en el campus
Herramientas	PostgreSQL 16+ · SQLite 3.37+ · Kiro y OpenCode
Puntaje	100 puntos · se aprueba con 60

Ciclo lectivo 2026 · Cátedra de Base de Datos II

De qué se trata

Es la historia del apunte, hecha con **tu Food Store**: el food truck vende en una feria sin señal. Vas a armar la **base de la tablet** en SQLite, **simular la sincronización** con PostgreSQL usando el semáforo, y **cerrar la capa de datos** del proyecto con un mapa, un plan de mantenimiento y una revisión con IA.

> **Regla de oro**
>
> **El móvil propone, el servidor dispone.** La tablet solo consulta y arma borradores. Precio, stock y ventas los decide PostgreSQL.

Consigna	Qué hacés	Paradas del apunte	Puntos
1 · La base de la tablet	Elegir qué datos viajan, crear las tablas SQLite y probarlas	1 a 4	30
2 · Sincronizar y resolver	Traer la instantánea del servidor, aplicar el semáforo y mostrar los casos	5 a 8	35
3 · Integrar y revisar	Mapa de los tres motores, plan de mantenimiento y code review con IA	9 y 10	20
4 · Kiro, bitácora y defensa	Spec en Kiro, registro del uso de IA, README y ensayo de defensa	10	15

> **Reglas que no se negocian**
>
> 1. Trabajar sobre una **copia** de la base, con **datos inventados**.
> 2. Nada de contraseñas, mails o datos personales reales en SQLite ni en lo que le pidas a la IA.
> 3. La tablet **nunca** cambia el stock ni el precio del servidor, ni confirma ventas: `SIMULADO_APROBADO` no es una venta.
> 4. Todo lo que generes con IA lo leés y lo probás antes de ejecutarlo; lo que no puedas explicar en la defensa cuenta como no hecho.

No se pide app de celular, interfaz ni programa en Python: **todo se puede hacer con SQL** en SQLite (con `sqlite3` o DB Browser for SQLite) y en PostgreSQL. Si preferís, podés usar Python para la simulación.

Consigna 1 · La base de la tablet (30 puntos)

> **Repasá en el apunte**
>
> Paradas 1 a 4: qué es SQLite, qué datos viajan, el esquema de la tablet y la transacción del borrador.

1.a Qué datos viajan (5 puntos)

Una tabla corta con las columnas de `producto` que viajan a la tablet y las que **no** viajan (por ejemplo, datos de usuarios). Dos o tres oraciones: por qué, y quién es la fuente de verdad.

1.b Las tablas (15 puntos)

Script `schema_sqlite.sql` que empiece con `PRAGMA foreign_keys = ON;` y cree estas cuatro tablas (`STRICT`), con las columnas mínimas:

Tabla	Columnas mínimas	Reglas que tiene que cumplir
`producto_cache`	id, nombre, precio_centavos, stock_visto, disponible, eliminado, fecha de copia	Precio en centavos (entero) y no negativo; disponible y eliminado solo 0 o 1
`pedido_borrador`	id_local (UUID), usuario, forma_pago, fecha	Forma de pago solo EFECTIVO, TARJETA o TRANSFERENCIA
`item_borrador`	id_local, producto_id, cantidad, precio_centavos_visto	Claves foráneas a las dos tablas anteriores; cantidad mayor que 0
`cola_sync`	solicitud_uuid (clave primaria), id_local, estado, motivo	Estado solo PENDIENTE, CONFLICTO, RECHAZADO o SIMULADO_APROBADO

1.c Probarla (10 puntos)

En `pruebas_sqlite.sql`, mostrá:

1. Un borrador completo (borrador + ítems + solicitud) guardado con `BEGIN … COMMIT`, y otra operación deshecha con `ROLLBACK`.
2. **Caso C1:** la consulta del catálogo disponible funciona sin conectarse al servidor.
3. **Caso C7:** una cantidad en 0 y un producto inexistente dan error y no dejan filas nuevas.

Consigna 2 · Sincronizar y resolver (35 puntos)

> **Repasá en el apunte**
> Paradas 5 a 8: la instantánea y las lápidas, el semáforo, los conflictos y la idempotencia.

2.a La instantánea (5 puntos)

En tu PostgreSQL de pruebas, creá la vista `v_snapshot_movil` de la parada 5 y exportala a CSV. Importala en SQLite en una tabla `snapshot_servidor` y cargá con ella `producto_cache`. Anotá la fecha de la copia.

Exportar e importar la instantánea

```
-- En psql (PostgreSQL de PRUEBAS)
\copy (SELECT * FROM v_snapshot_movil) TO 'servidor_snapshot.csv' CSV HEADER

-- En sqlite3 (o con "Importar CSV" de DB Browser)
.import --csv --skip 1 servidor_snapshot.csv snapshot_servidor
```

2.b El semáforo (15 puntos)

Una consulta (o un script) que compare cada solicitud `PENDIENTE` con `snapshot_servidor` y guarde en `cola_sync` el **estado** y el **motivo**:

- **Rojo · RECHAZADO:** algún producto no existe, no está disponible, fue dado de baja o no alcanza el stock.
- **Amarillo · CONFLICTO:** no hay rojo, pero cambió algún precio.

- **Verde · SIMULADO_APROBADO:** todo coincide.

Solo se evalúan las solicitudes `PENDIENTE` (así, repetir la simulación no cambia nada). **No se modifica nada en PostgreSQL.**

> **Pista**
>
> Agrupá por solicitud y contá los ítems con problemas. Por ejemplo: `CASE WHEN SUM(i.cantidad > s.stock OR …) > 0 THEN 'RECHAZADO' WHEN SUM(i.precio_centavos_visto <> s.precio_centavos) > 0 THEN 'CONFLICTO' ELSE 'SIMULADO_APROBADO' END`, con un `LEFT JOIN` de `item_borrador` a `snapshot_servidor`. En el CSV de `psql`, los booleanos vienen como `t` y `f`.

2.c Los casos (15 puntos · 3 cada uno)

Prepará cada situación cambiando datos **en la copia del servidor**, exportá una instantánea nueva y corré el semáforo:

Caso	Qué preparás	Resultado esperado
C2	Un borrador; después subís ese precio en el servidor	`CONFLICTO`. Luego el cliente acepta: actualizás el precio del borrador y una solicitud nueva da `SIMULADO_APROBADO`
C3	Un borrador por más unidades de las que quedan	`RECHAZADO`
C4	Un borrador con un producto que después das de baja (baja lógica)	`RECHAZADO`; al recargar el catálogo, el producto ya no aparece pero la fila queda
C5	Un borrador sin cambios	`SIMULADO_APROBADO`
C6	Reenviar la misma solicitud (mismo UUID)	No se duplica: el UUID repetido da error de clave primaria y repetir la simulación no reevalúa

Para cada caso: el comando, el resultado y una o dos oraciones que lo expliquen. Al final, una consulta en PostgreSQL que muestre que la cantidad de pedidos y el stock **no cambiaron**. En C6, agregá una oración sobre dónde iría la protección real en un sistema de verdad (parada 8).

Consigna 3 · Integrar y revisar (20 puntos)

> **Repasá en el apunte**
>
> Paradas 9 y 10: los tres motores y el code review con IA.

3.a Mapa de la capa de datos (5 puntos)

Una tabla o un dibujo con los tres motores de tu proyecto (PostgreSQL, tu parte de MongoDB de la Unidad 7 y SQLite): qué guarda cada uno y quién manda sobre cada dato.

3.b Plan de mantenimiento (5 puntos)

Respondé en pocas líneas, como agregado al plan de la Unidad 6:

- ¿Qué se respalda y cómo se prueba la restauración?
- Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

- ¿Cada cuánto y quién revisa las solicitudes en `CONFLICTO` o `RECHAZADO`?

3.c Code review con IA (10 puntos)

Pedile a OpenCode, en el agente **Plan**, que revise **todos** los scripts de la cursada. En una tabla, anotá **cada hallazgo** con su decisión (aceptado, corregido o descartado) y el motivo. Desarrollá al menos **tres**, de al menos **dos unidades** distintas, indicando cómo lo comprobaste en la base. **Al menos uno tiene que estar descartado** con fundamento.

Consigna 4 · Kiro, bitácora y defensa (15 puntos)

4.a Spec en Kiro (5 puntos)

Antes de generar código, escribí en Kiro `requirements.md` con **cuatro requisitos** en notación EARS, cada uno con el caso que lo prueba, y un `design.md` y `tasks.md` breves. El esquema y la simulación se **generan con OpenCode** a partir de esas tareas.

Ejemplo

```
REQ-01  WHEN el precio actual es distinto del precio visto
        THE SYSTEM SHALL marcar la solicitud como CONFLICTO.        Se prueba con: C2
```

4.b Bitácora de IA y DUIA (5 puntos)

Al menos tres entradas: qué le pediste a la IA, qué aceptaste, **qué descartaste y por qué**. Más la Declaración de Uso de IA de la cátedra.

4.c README y ensayo de defensa (5 puntos)

Un `README.md` con el orden para crear todo desde cero. En la clase de la semana 13, un ensayo de 10 minutos: el mapa, un caso en vivo y un hallazgo de la IA.

Qué se entrega

Un `.zip` (o el enlace al repositorio con la etiqueta `tp8`) en el campus, antes del cierre de la semana 13. **No hay entregas tardías**: la semana 14 es el segundo parcial.

```
food-store/
├── README.md
├── movil/
│   ├── schema_sqlite.sql       consigna 1.b
│   ├── pruebas_sqlite.sql      consigna 1.c
│   ├── semaforo.sql            consigna 2.b (o simulador.py)
│   ├── servidor_snapshot.csv   consigna 2.a
│   └── foodstore_movil.db
└── docs/
    ├── informe.pdf             1.a, 2.c, 3.a, 3.b y 3.c (máximo 6 páginas)
    ├── kiro/                   requirements.md · design.md · tasks.md
    ├── bitacora_ia.md
    └── DUIA.md
```

Cómo se califica

Consigna	Puntos	Está bien si…
1 · La base de la tablet	30	El script crea la base desde cero, con claves foráneas activas, centavos, `CHECK` y transacciones probadas
2 · Sincronizar y resolver	35	Los casos C2 a C6 dan el resultado esperado y PostgreSQL no cambió
3 · Integrar y revisar	20	El mapa dice quién manda, el plan contempla las tablets y cada hallazgo tiene decisión
4 · Kiro, bitácora y defensa	15	Cuatro requisitos ligados a casos, bitácora con algo descartado, README que funciona
Total	100	Nota = puntaje ÷ 10 · se aprueba con 60 puntos (nota 6)

Errores que más se descuentan: cambiar el stock o el precio del servidor desde la tablet; presentar `SIMULADO_APROBADO` como venta; entregar capturas sin scripts; aceptar todo lo que dijo la IA sin comprobarlo.